LIO-DPC: Accurate and Fast LiDAR-Inertial Odometry with Dynamic Pose Chain

Yuexin Mu[1], Ao Ren[1], Duo Liu[1], Zihao Zhang[1], Haojie Lu[1], Longyi Zhou[1]
Huachen Tan[1], Kan Zhong[1], Yujuan Tan[1], and Chaoxia Qin[1]

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11345708

***Abstract*—LiDAR-inertial odometry is widely used in robotics navigation, autonomous driving, and drone operation to provide precise, low-latency motion estimation. Filter-based methods are fast but suffer from significant cumulative errors. Graph optimization methods reduce cumulative errors through loop closure detection but are computationally expensive. In this work, we propose LIO-DPC, a framework that combines the benefits of the filter-based approach and graph-based approach. First, we propose a dynamic pose chain optimization method. It generates an initial pose chain using the fast filter. This is followed by applying computationally efficient local graph optimization to a set of local pose chains to generate refined relative poses, which are then used to update the motion estimation. Second, we propose a loop sparsification approach to select representative loops that are both temporally and spatially proximate, to reduce the computational complexity in graph optimization and minimize loop errors. Extensive experiments demonstrate that LIO-DPC achieves real-time performance and outperforms state-of-the-art methods in accuracy.**

***Index Terms*—LiDAR-inertial odometry, SLAM, state estimation**

I. Introduction

LiDAR-inertial odometry is a method that combines LiDAR data and inertial measurement unit (IMU) data for self-motion estimation [1]. Since LiDAR can directly acquire depth information from the environment, LiDAR-inertial odometry is not affected by ambient lighting conditions [2]. Moreover, integrating high-frequency motion measurements from the IMU can effectively address issues of motion distortion in LiDAR point clouds and degradation problems caused by indistinct environmental geometric information [3]. Therefore, LiDAR-inertial odometry is widely employed in intelligent robots, autonomous vehicles, unmanned aerial vehicles, and other terminal devices, providing accurate and low-latency self-motion estimation information.

In current research on LiDAR-inertial odometry, methods can mainly be divided into two categories: filtering-based methods and graph-based methods [4], [5]. Filtering-based methods, including FAST-LIO [6] and LINS [7], utilize the previous state and inputs of the sensors to estimate the current state of the robot in real time. However, these methods suffer from cumulative errors that increase over time or with the distance traveled, potentially leading to inaccurate or even failed state estimations. Graph-based methods, including LIO-SAM [8] and LIPS [9], optimize the state of the robot by constructing factor graphs, typically using measurement factors of the LiDAR, prediction factors of the IMU, and factors of the loop closures. The introduction of loop closure factors can effectively correct cumulative errors along the entire loop trajectory, making graph-based methods more accurate in large-scale environments and long-term state estimation. However, as the scale of optimization increases, the computational load of these methods also rises significantly, leading to decreased real-time performance.

A recent study [10] has explored combining filtering and graph-based approaches by introducing graph optimization after loop closure detection. Although this mitigates cumulative errors, it introduces a significant delay, as the filter state cannot be updated until graph optimization is complete. This trade-off highlights a key challenge: how to retain the fast updates of filtering methods while leveraging the accuracy improvements of graph optimization. Addressing this issue is crucial for advancing LIO in real-time applications.

In this work, we propose LIO-DPC, a novel LIO framework that addresses the aforementioned challenges through two key innovations: dynamic pose chain and loop sparsification. We deem that the key to combining the advantages of the filtering methods with the graph optimization methods lies in decoupling the two and reducing the computation complexity of the graph optimization, allowing the two methods to operate in parallel. To achieve this, we propose a dynamic pose chain—a sequence of relative poses that can be dynamically updated. Initially, filtering methods provide high-frequency relative pose estimates to construct the chain. Upon detecting a loop closure, the system builds a local pose graph using both the loop closure and the dynamic pose chain. This graph is optimized to refine the relative pose estimates, which are subsequently used to update the chain. This design ensures real-time performance while reducing cumulative errors through loop closure correction.

Furthermore, the system often accumulates multiple loop closures upon entering the pose graph optimization stage. These loop closures are spatially and temporally closed and hold errors. Optimizing all of these loop closures not only consumes excessive computational resources but also risks introducing additional inaccuracies. To address these issues, we propose the loop sparsification strategy to select only the most impactful and high-quality loop closures for optimiza-

[1]The authors are with the Chongqing University, Chongqing, China, yuexinmu@stu.cqu.edu.cn, ren.ao@cqu.edu.cn.
Corresponding authors: Ao Ren and Duo Liu.
The code will be available at https://github.com/YuexinMu/lio_dpc.git.

tion. Specifically, an evaluation function is designed to account for both the scale of the pose graph optimization and their respective registration scores. Ultimately, the loop closure with the highest comprehensive score is selected for local pose graph optimization, thereby reducing computational overhead and minimizing errors.

Our contributions can be summarized as follows:

- We propose a dynamic pose chain method that enables the parallel operation of filtering and graph optimization. While maintaining the high-frequency pose updates of filters, it leverages loop closure detection and graph optimization to enhance system accuracy.
- We propose a loop quality evaluation metric to assess temporally and spatially adjacent loops, aiming to select high-quality loop closures for optimization.
- We develop a LiDAR-inertial odometry system (LIO-DPC), and experimental results on multiple datasets demonstrate that our method achieves a significant improvement in accuracy compared to state-of-the-art approaches while maintaining high real-time performance.

II. Related Works

The LIO system estimates the state of robots by integrating relative transformations between consecutive LiDAR scans and IMU measurements. The iterative closest point (ICP) method [11] iteratively minimizes the distance between point clouds. However, motion distortion in LiDAR data due to sensor movement can cause significant drift if relying solely on point cloud matching. Therefore, LiDAR is typically combined with other sensors like IMU for state estimation [12]–[16]. Current LIO systems employ either filtering-based or graph optimization-based methods to fuse LiDAR and IMU data for robot state estimation.

Several works have employed Kalman Filters to fuse LiDAR and IMU data for state estimation [17]–[22]. Tang et al. [23] used an Extended Kalman Filter (EKF), while Zhen et al. [24] adopted an Error State Kalman Filter (ESKF) to enhance robustness and accuracy. LINS [7] utilized an Iterative Error State Kalman Filter (IESKF) for improved speed over graph-based methods but still faced computational burdens, especially when calculating the Kalman gain due to numerous LiDAR measurements. FAST-LIO [6] addressed this by introducing a novel Kalman gain formulation. FAST-LIO2 [2] further improved accuracy and speed by eliminating feature extraction and directly registering raw LiDAR data using the ikd-Tree structure. Despite these computational improvements, filtering-based LiDAR-inertial odometry systems still cannot effectively mitigate error accumulation during operation.

Graph-based methods enhance odometry performance by incorporating various factors into factor graphs, though this increases computational load. LIPS [9] includes continuous IMU pre-integration and 3D plane factors from LiDAR measurements. IN2LAMA [25] uses upsampled pre-integrated IMU data to undistort LiDAR scans, forming a batch manifold optimization with LiDAR factors, IMU bias factors, and sensor time offsets. LIO-SAM [8] employs IMU pre-integration to undistort LiDAR data for joint optimization in a factor graph. LILI-OM [26] utilizes hierarchical pose graph optimization and introduces a novel feature extraction method for solid-state LiDARs with irregular scanning patterns. Additionally, Setterfield et al. [27] directly integrate feature correspondences from LiDAR measurements into the factor graph.

To leverage the speed of filtering methods and the error reduction benefits of graph optimization with loop closure detection, Tang et al. [10] attempted to construct factor graphs and update filter states upon loop detection. However, the slow graph optimization process negates the real-time advantages of filtering due to the required waiting time.

III. Method

A. System Overview

The system workflow of LIO-DPC is shown in Fig.1. The system first processes LiDAR and IMU data through a filter to quickly estimate the pose of the current moment relative to the previous one. The incremental update module uses this relative pose along with the existing dynamic pose chain to perform incremental updates, providing high-frequency pose estimates while constructing the initial dynamic pose chain. When the loop detection module identifies a loop closure, it passes it to the loop sparsification module, which selects the highest-scoring loop based on the loop scores and sends it to the local pose graph optimization module. The local pose graph optimization module constructs a local pose graph, optimizes it using the least squares method, and computes the optimized relative poses. Subsequently, the batch update module utilizes these optimized relative poses to perform batch updates on the dynamic pose chain, obtaining more accurate pose estimates. Since both incremental updates and batch updates operate only on the dynamic pose chain, they do not interfere with each other and can be executed entirely in parallel. By employing this approach, LIO-DPC achieves fast and precise pose estimation.

B. Dynamic Pose Chain

The dynamic pose chain is a sequentially updated set of relative poses that decouples filtering methods from graph optimization methods, allowing them to operate independently while effectively combining their respective advantages. Initially, the filtering method performs incremental updates, constructing the initial dynamic pose chain and providing real-time pose estimates. When the system detects a loop closure, it constructs a local pose graph based on the dynamic pose chain for optimization. This process yields more accurate local poses and performs batch updates on the dynamic pose chain, reducing accumulated errors and enhancing the accuracy of pose estimation. Since the filtering method and the graph optimization method operate independently on the dynamic pose chain, they do not interfere with each other. Consequently, the system update frequency matches the filtering method, while still providing substantial accuracy improvements.

We can represent the pose of the robot using $\mathbf{x}$:

$$\mathbf{x} = \mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3); \quad \mathbf{R} \in SO(3), t \in \mathbb{R}^3 \tag{1}$$

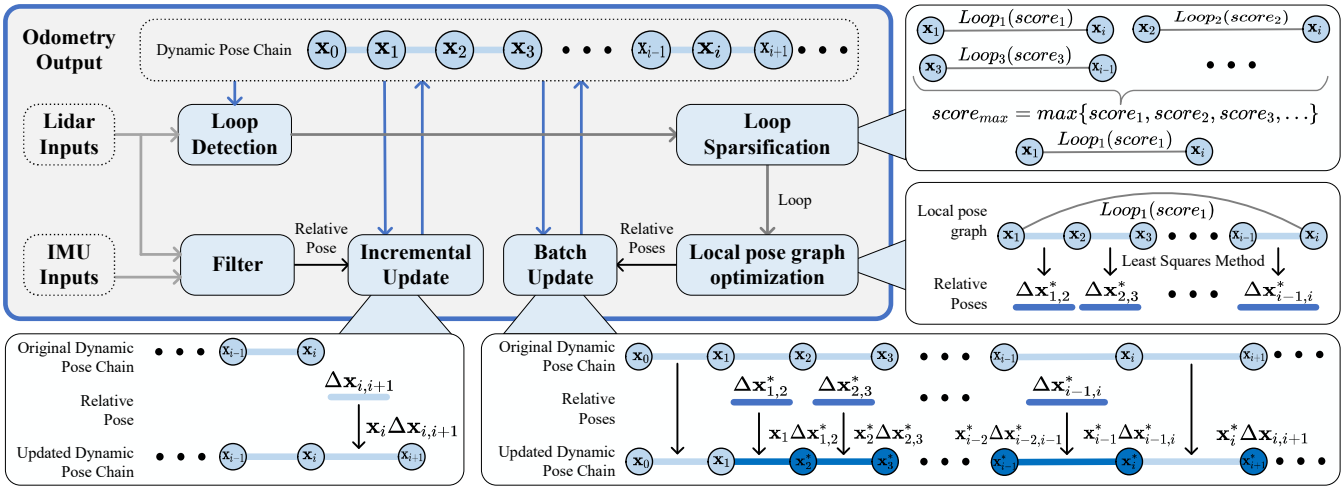

Fig. 1. System overview of LIO-DPC.

where $\mathbf{R}$ is the rotation matrix, and $\mathbf{t}$ is the position vector. $SE(3)$ denotes the special Euclidean group.

We denote the relative pose of the robot by $\Delta\mathbf{x}$. Assuming the poses of the robot at times i and j are known as $\mathbf{x}_i$ and $\mathbf{x}_j$, the relative pose from time i to time j is represented as:

$$\Delta\mathbf{x}_{i,j} = \mathbf{x}_i^{-1}\mathbf{x}_j = \begin{bmatrix} \mathbf{R}_i^\top \mathbf{R}_j & \mathbf{R}_i^\top(\mathbf{t}_j - \mathbf{t}_i) \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{2}$$

where $\mathbf{R}_i^\top$ is the transpose of $\mathbf{R}_i$.

We define the dynamic pose chain $\mathcal{C}_{0,n}$ as:

$$\mathcal{C}_{0,n} = (\mathbf{x}_0, \{\Delta\mathbf{x}_{0,1}, \Delta\mathbf{x}_{1,2}, \ldots, \Delta\mathbf{x}_{n-1,n}\}); \quad n \in \mathbb{N} \tag{3}$$

where $\mathbf{x}_0$ is the starting pose of the dynamic pose chain, and the set$\{\Delta\mathbf{x}_{0,1}, \Delta\mathbf{x}_{1,2}, \ldots, \Delta\mathbf{x}_{n-1,n}\}$ represents the sequence of continuous relative poses from $\mathbf{x}_0$ to $\mathbf{x}_n$. The initial dynamic pose chain is defined as $\mathcal{C}_{0,0} = (\mathbf{x}_i, \emptyset)$, indicating only the initial pose with no sequence of relative poses.

1) Incremental Update: Incremental updating involves adding new relative poses to the existing dynamic pose chain and computing the absolute pose at the latest time relative to the initial time. Since this process increases the length of the dynamic pose chain, it is called incremental updating. The pose estimated directly by the filter contains accumulated historical errors; therefore, we only use the relative pose estimated by the filter from the previous moment to the current moment. Because this time interval is short and the error is relatively small. Then, we calculate the current pose based on the previous pose in the dynamic pose chain and the relative pose. Since the pose in the dynamic pose chain can improve accuracy through stock updates, the pose estimation obtained in this way is more precise. Additionally, when the system detects a loop closure, the dynamic pose chain constructed through incremental updating can quickly build a local pose graph and optimize its relative pose. This not only enhances accuracy but also does not affect the update speed of the filter.

Assuming the system receives new LiDAR and IMU data, the relative pose calculated by the filter is $\Delta\mathbf{x}_{n-1,n}$. According to the definition of the relative pose chain in (3), we can add the new relative pose to the existing dynamic pose chain, completing the incremental update:

$$\mathcal{C}_{0,n} = \mathcal{C}_{0,n-1} + \Delta\mathbf{x}_{n-1,n} \tag{4}$$

Simultaneously, computing the absolute pose at the current moment relative to the initial moment can be achieved by:

$$\mathbf{x}_n = \mathbf{x}_{n-1}\Delta\mathbf{x}_{n-1,n} \tag{5}$$

LIO-DPC uses (4) to perform incremental updates on the dynamic pose chain. This allows for independent optimization of existing relative poses in the chain without impacting filter updates. Using (5), we ensure that the overall real-time performance of the system and the speed of the filtering updates remain consistent, thereby achieving real-time performance comparable to filtering methods.

2) Local Pose Graph Optimization: Although the dynamic pose chain integrates relative poses generated by filtering, it can accumulate significant errors over time. To minimize these errors, we employ local pose graph optimization to obtain more accurate relative poses. Specifically, we construct a local pose graph using loops and segments of the dynamic pose chain and optimize the relative poses within it. These optimized relative poses then serve as inputs to the batch update module.

Assuming the local pose graph optimization module detects a loop from time i to j, where $i, j \in [1, n]$ and $i < j$, we represent the poses of the robot as nodes and the relative poses between them as edges. Using the dynamic pose chain $\mathcal{C}_{0,n}$ and the loop edge $\Delta\mathbf{x}_{i,j}$, we can construct a pose graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ that includes $m(m = j - i + 1)$ nodes and m edges. The sets of nodes $\mathcal{V}$ and edges $\mathcal{E}$ are defined as:

$$\begin{cases} \mathcal{V} = \{\mathbf{x}_i, \mathbf{x}_{i+1}, \ldots, \mathbf{x}_{j-1}, \mathbf{x}_j\} \\ \mathcal{E} = \{\Delta\mathbf{x}_{i,i+1}, \ldots, \Delta\mathbf{x}_{j-1,j}, \Delta\mathbf{x}_{j,i}\} \end{cases} \tag{6}$$

Since $\mathcal{G}$ includes loop constraints, it can be optimized using the least squares method, with the error function defined as:

$$\mathbf{F}(\mathbf{x}) = \sum_{k,l \in \mathcal{G}} \mathbf{e}(\mathbf{x}_k, \mathbf{x}_l, \Delta\mathbf{x}_{k,l})^\top \Omega_{k,l} \mathbf{e}(\mathbf{x}_k, \mathbf{x}_l, \Delta\mathbf{x}_{k,l}) \tag{7}$$

where $\Omega_{k,l}$ represents the information matrix between nodes k and l, and $\mathbf{e}(\mathbf{x}_k, \mathbf{x}_l, \Delta\mathbf{x}_{k,l})$ represents the error between the edge $\Delta\mathbf{x}_{k,l}$ and node poses. To simplify notation, we denote

the error function as $\mathbf{e}_{k,l}(\mathbf{x})$. The optimization objective is to minimize this error function:

$$\begin{aligned}\mathbf{x}^* &= \underset{\mathbf{x}}{\operatorname{argmin}} \mathbf{F}(\mathbf{x}) \\ &= \underset{\mathbf{x}}{\operatorname{argmin}} \sum_{(k,l)\in\mathcal{G}} \mathbf{e}_{k,l}(\mathbf{x})^\top \Omega_{k,l} \mathbf{e}_{k,l}(\mathbf{x})\end{aligned} \tag{8}$$

With the starting point $\mathbf{x}_i$ fixed, the optimized set of node poses can be obtained through least squares optimization:

$$\mathcal{V}^* = \{\mathbf{x}_i, \mathbf{x}^*_{i+1}, \ldots, \mathbf{x}^*_{j-1}, \mathbf{x}^*_j\} \tag{9}$$

Correspondingly, the optimized relative poses between adjacent nodes can be computed:

$$\mathcal{E}^* = \{\Delta\mathbf{x}^*_{i,i+1}, \ldots, \Delta\mathbf{x}^*_{j-1,j}\} \tag{10}$$

The incorporation of loop constraints means these optimized relative poses have less cumulative error compared to the original poses before optimization.

3) Batch Update: We use the relative poses obtained from the local pose graph optimization module to update the existing dynamic pose chain and compute the absolute pose of the latest moment relative to the initial moment. We refer to this process as batch updating, which modifies the relative poses in the existing dynamic pose chain to effectively reduce cumulative errors. Since the batch update process operates solely on the dynamic pose chain, it does not affect the filtering process. Thus, it enhances the accuracy of pose estimation for the system while maintaining the speed of filter updates.

According to the definition of the dynamic pose chain in (3), given the chains $\mathcal{C}_{a,b}$ and $\mathcal{C}_{b,c}$:

$$\begin{cases} \mathcal{C}_{a,b} = (\mathbf{x}_a, \Delta\mathbf{x}_{a,a+1}, \ldots, \Delta\mathbf{x}_{b-1,b}) \\ \mathcal{C}_{b,c} = (\mathbf{x}_b, \Delta\mathbf{x}_{b,b+1}, \ldots, \Delta\mathbf{x}_{c-1,c}) \end{cases} \tag{11}$$

they can be merged to form:

$$\mathcal{C}_{a,c} = (\mathbf{x}_a, \Delta\mathbf{x}_{a,a+1}, \ldots, \Delta\mathbf{x}_{b,b+1}, \ldots, \Delta\mathbf{x}_{c-1,c})$$

We define the merging operation as:

$$\mathcal{C}_{a,c} = \mathcal{C}_{a,b} \oplus \mathcal{C}_{b,c}; \quad a \le b \le c \tag{12}$$

Similarly, we can define the disassembly formula for the dynamic pose chain:

$$\mathcal{C}_{a,b} = \mathcal{C}_{a,c} \ominus \mathcal{C}_{b,c}; \quad a \le b \le c \tag{13}$$

Based on the optimized relative poses from (10), we can further construct the dynamic pose chain between moments i and j as:

$$\mathcal{C}^*_{i,j} = (\mathbf{x}_i, \{\Delta\mathbf{x}^*_{i,i+1}, \ldots, \Delta\mathbf{x}^*_{j-1,j}\}) \tag{14}$$

Using (12) and (13), we can perform a batch update on the original dynamic pose chain:

$$\begin{aligned}\mathcal{C}^*_{0,n} &= \mathcal{C}_{0,n} \ \ominus \ \mathcal{C}_{i,j} \ \oplus \ \mathcal{C}^*_{i,j} \\ &= (\mathbf{x}_0, \ \{\Delta\mathbf{x}_{0,1}, \ \ldots, \ \Delta\mathbf{x}_{i-1,i}, \\ &\qquad \Delta\mathbf{x}^*_{i,i+1}, \ \ldots, \ \Delta\mathbf{x}^*_{j-1,j}, \\ &\qquad \Delta\mathbf{x}_{j,j+1}, \ \ldots, \ \Delta\mathbf{x}_{n-1,n}\})\end{aligned} \tag{15}$$

where $\mathcal{C}_{0,n}$ and $\mathcal{C}_{i,j}$ respectively represent the original dynamic pose chain and the local dynamic pose chain. $\mathcal{C}^*_{0,n}$ represents the dynamic pose chain after batch updating.

Consequently, the latest pose of the robot relative to the initial moment, $\mathbf{x}_n$, can be calculated using the recursive relationship:

$$\mathbf{x}_n = \mathbf{x}_{n-1}\Delta\mathbf{x}_{n-1,n} \tag{16}$$

By expanding this recursive relationship, $\mathbf{x}_n$ can be expressed as a product of the initial pose and a series of relative pose transformations:

$$\begin{aligned}\mathbf{x}_n &= \mathbf{x}_0\Delta\mathbf{x}_{0,1}\Delta\mathbf{x}_{1,2}\cdots\Delta\mathbf{x}_{n-1,n} \\ &= \mathbf{x}_0 \prod_{k=1}^{n} \Delta\mathbf{x}_{k-1,k}\end{aligned} \tag{17}$$

Thus, after the batch update, the optimized current pose of the robot can be computed using (17) and updated dynamic pose chain:

$$\mathbf{x}^*_n = \mathbf{x}_0 \prod_{k=1}^{i} \Delta\mathbf{x}_{k-1,k} \prod_{k=i+1}^{j} \Delta\mathbf{x}^*_{k-1,k} \prod_{k=j+1}^{n} \Delta\mathbf{x}_{k-1,k} \tag{18}$$

It is important to note that due to the non-commutativity of pose transformations, the symbol $\prod$ in the formulas represents the right multiplication of relative poses $\Delta\mathbf{x}_{k-1,k}$ in chronological order from $k = 1$ to $k = n$.

LIO-DPC reduces cumulative errors by applying local pose graph optimization and batch updates to the dynamic pose chain, enhancing pose accuracy. As this process operates independently of the filter, it preserves the filter's real-time update capability. This design improves overall precision without sacrificing real-time performance.

C. Loop Sparsification

During the batch updating process of the dynamic pose chain, multiple loop closures may exist when entering pose graph optimization due to the high speed of loop closure detection. Since these loop closures are temporally and spatially close and may contain errors, optimizing all of them does not significantly improve the results but consumes excessive computational resources and introduces additional errors. Therefore, we aim to optimize only the loop closure that contributes the most to the overall optimization and has the highest quality. Specifically, we evaluate loop closures using two metrics: their impact on the overall optimization scale and their own point cloud registration scores.

We can represent a loop closure as:

$$\mathcal{L} = (\mathbf{r}, \mathbf{p}, \Delta\mathbf{x}) \tag{19}$$

where $\mathbf{r}$ is the time difference between the current pose and the previous pose involved in the loop closure, $\mathbf{p}$ is the point cloud registration score, and $\Delta\mathbf{x}$ is the relative pose transformation associated with the loop closure. Larger $\mathbf{r}$ values correspond to longer loop closure paths, resulting in more extensive pose graphs for optimization. Higher $\mathbf{p}$ values indicate superior registration quality with reduced inherent loop closure errors.

Therefore, we comprehensively evaluate loop closures using these two metrics, $\mathbf{r}$ and $\mathbf{p}$.

Suppose there are m loop closure candidates when entering the graph optimization module. Since $\mathbf{r}$ and $\mathbf{p}$ have different orders of magnitude, we need to normalize $\mathbf{r}$ and $\mathbf{p}$ across these m loop closures.

$$\begin{cases} \bar{\mathbf{r}}_i = \frac{\mathbf{r}_i}{\sum_{j=1}^{m} \mathbf{r}_j}; \quad i \in [1, m] \\ \bar{\mathbf{p}}_i = \frac{\mathbf{p}_i}{\sum_{j=1}^{m} \mathbf{p}_j}; \quad i \in [1, m] \end{cases} \tag{20}$$

where $\bar{\mathbf{r}}_i$ and $\bar{\mathbf{p}}i$ represent the normalized pose graph optimization scale score and the normalized loop closure registration score for the i-th loop closure, respectively. The normalization is performed using the sums $\sum_{j=1}^{m} \mathbf{r}_j$ and $\sum_{j=1}^{m} \mathbf{p}_j$, which are the total scores over all m loop closures. We propose a weighting coefficient $\mathbf{k} \in [0, 1]$ to balance the contributions of the two scores and compute the final score for each loop closure as follows:

$$\begin{cases} \mathcal{S}_i = k * \bar{\mathbf{r}}_i + (1 - k) * \bar{\mathbf{p}}_i; \quad k \in [0, 1], i \in [1, m] \\ \mathcal{S}_{last} = max\{\mathcal{S}_1, \mathcal{S}_2, \ldots, \mathcal{S}_m\} \end{cases} \tag{21}$$

here $\mathcal{S}_i$ represents the final score of the i-th loop closure.

Using this as a criterion, we select the loop closure with the highest score, $\mathcal{S}_{\text{last}}$, from the m loop closures and send its corresponding loop closure to the final pose graph optimization module.

IV. Experiments

A. Experimental Setup

Datasets. We conducted experiments using four publicly available datasets from UTBM [28], URBAN [29], NCLT [30], and LIOSAM (from the LIO-SAM [8] dataset), with detailed information provided in Table I. These challenging datasets are widely used for benchmarking state-of-the-art methods. Due to variability in ground truth data caused by factors such as weather conditions and GPS quality, we selected 13 sequences with reliable ground-truth trajectories from these datasets. These datasets cover various scenarios, including autonomous driving, urban environments, and campus settings, with the longest distance reaching 5.1 km and the longest duration being 111 minutes.

TABLE I
Details of All Dataset Sequences

	Name	Duration (min:sec)	Distance (km)
utbm1	20180719	15:26	4.98
utbm2	20180717	15:59	4.99
utbm3	20180713	16:59	5.03
utbm4	20190418	11:59	5.11
urban1	20190428	8:07	2.00
urban2	20210517	13:05	3.64
urban3	20210521	25:36	4.51
nclt1	20120115	111:46	4.01
nclt2	20120429	43:17	1.86
nclt3	20120511	84:32	3.13
nclt4	20120615	55:10	1.62
nclt5	20130110	17:02	0.26
lio-sam1	park	9:11	0.66

Implementation Details. We evaluated the accuracy and efficiency of our proposed LIO-DPC method by comparing it with state-of-the-art LiDAR-inertial odometry techniques. The comparison includes filtering-based methods FAST-LIO2 [2] and FASTER-LIO [31], as well as factor graph optimization-based methods LILI-OM [26] and LIO-SAM [8]. All experiments were performed on a computer equipped with an AMD R9-5950X CPU (16 cores at 3.4GHz) and 128GB of RAM. Performance was evaluated using the Root Mean Square Error (RMSE) of the Absolute Pose Error (APE) over the entire trajectory. Notably, LIO-SAM could not operate on UTBM datasets due to missing attitude quaternion data (denoted by $-$), and significant drift led to algorithm failures in some sequences (denoted by $\times$).

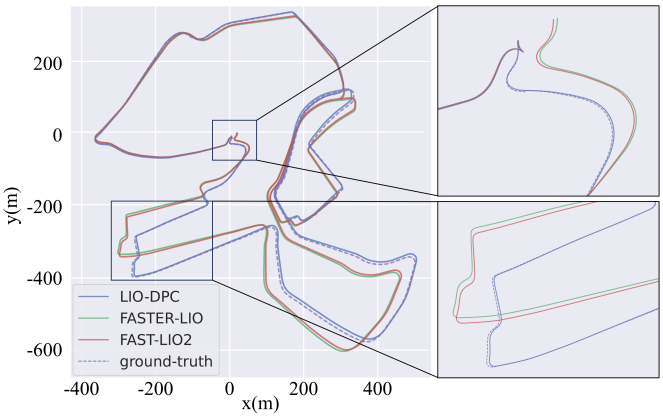

Fig. 2. Ground-Truth and Estimated Trajectories on the UTBM Dataset: Dashed Lines Indicate Ground-Truth, Solid Lines Represent Estimated Results.

B. Accuracy Evaluation

The RMSE results, shown in Table II, demonstrate that LIO-DPC consistently achieves lower RMSE compared to other state-of-the-art methods across all datasets. Compared to FAST-LIO2 and FASTER-LIO, LIO-DPC utilizes batch updates, significantly reducing cumulative errors and improving system accuracy. Additionally, unlike LILI-OM, LIO-DPC employs loop sparsification to filter out low-quality loop closures, effectively minimizing loop-related errors.

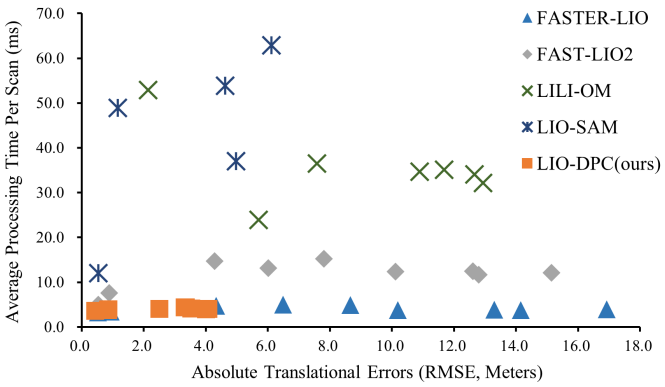

Fig. 3. Comparison of Absolute Translational Errors (RMSE, Meters) and Average Processing Time Per Scan (Milliseconds).

A trajectory comparison within the UTBM dataset sequence is shown in Fig.2, highlighting the significant drift experienced

TABLE II
ABSOLUTE TRANSLATIONAL ERRORS (RMSE, METERS) FOR SEQUENCES WITH HIGH-QUALITY GROUND TRUTH

	utbm1	utbm2	utbm3	utbm4	urban1	urban2	urban3	nclt1	nclt2	nclt3	nclt4	nclt5	liosam1
FAST-LIO2	15.15	12.61	12.80	10.11	4.28	7.81	6.01	1.89	1.56	2.51	2.00	0.89	0.53
FASTER-LIO	16.93	13.30	14.15	10.19	4.33	8.66	6.49	2.06	1.34	2.27	1.79	0.93	0.52
LILI-OM	10.89	12.66	12.94	11.68	35.51	7.58	5.70	×	×	×	×	2.14	21.86
LIO-SAM	–	–	–	–	1.16	6.12	4.62	×	×	×	×	4.98	0.54
LIO-DPC(ours)	**3.36**	**3.53**	**3.32**	**2.50**	**0.86**	**4.08**	**4.01**	**1.88**	**1.30**	**1.41**	**1.57**	**0.85**	**0.44**

TABLE III
AVERAGE PROCESSING TIME PER SCAN (MILLISECONDS)

	utbm1	utbm2	utbm3	utbm4	urban1	urban2	urban3	nclt1	nclt2	nclt3	nclt4	nclt5	liosam1
FAST-LIO2	12.14	12.47	11.70	12.37	14.67	15.18	13.10	9.24	9.38	8.80	9.77	7.57	4.95
FASTER-LIO	**3.96**	**3.81**	**3.75**	**3.80**	4.76	4.88	4.95	4.82	**4.41**	**4.00**	**4.17**	**3.42**	**3.26**
LILI-OM	34.70	34.05	32.16	35.14	36.66	36.47	23.93	×	×	×	×	52.89	53.88
LIO-SAM	–	–	–	–	48.88	62.88	53.86	×	×	×	×	37.01	12.02
LIO-DPC(ours)	4.35	4.14	4.33	4.04	**3.98**	**4.04**	**3.96**	**4.40**	4.43	4.26	4.41	3.74	3.55

by FAST-LIO2 and FASTER-LIO during operation. In contrast, LIO-DPC effectively mitigates these issues through loop closure detection and graph optimization, which systematically eliminate cumulative errors and maintain higher accuracy.

C. Processing Time Evaluation

As shown in Table III, we evaluate processing performance by measuring the average time per LiDAR frame. Compared to LILI-OM and LIO-SAM, LIO-DPC leverages a dynamic pose chain with incremental updates, achieving significantly better real-time performance. Despite integrating loop closure and local pose graph optimization, LIO-DPC maintains a processing speed comparable to the state-of-the-art filtering method FASTER-LIO. Moreover, batch optimization and loop closure allow LIO-DPC to achieve notably higher accuracy than FASTER-LIO. The dynamic pose chain enables parallel filtering and optimization, ensuring both efficiency and accuracy. Fig. 3 further illustrates that LIO-DPC achieves competitive speed while significantly outperforming other methods in accuracy across various datasets.

TABLE IV
ACCURACY EVALUATION ABLATION EXPERIMENT(RMSE, METERS)

	LIO-DPC	LIO-DPC (without sparsification)	LIO-DPC (without optimization)
utbm1	**3.36**	7.36	17.00
utbm2	**3.53**	4.95	13.26
utbm3	**3.32**	3.41	14.10
utbm4	**2.50**	5.72	11.80
urban1	**0.86**	1.46	4.58
urban2	**4.08**	6.19	8.40
urban3	**4.01**	5.01	6.89

D. Ablation Experiments

In this section, we conduct two ablation studies within the LIO-DPC framework to evaluate the effects of loop closure detection and loop sparsification on system accuracy and computational efficiency. We define LIO-DPC (without optimization) as the system without loop detection and local pose graph construction, and LIO-DPC (without sparsification) as the system without loop sparsification. Experiments were performed on the UTBM and URBAN datasets using these variants alongside the original LIO-DPC method. The accuracy results are presented in Table IV, and the total graph optimization time results are shown in Fig.4.

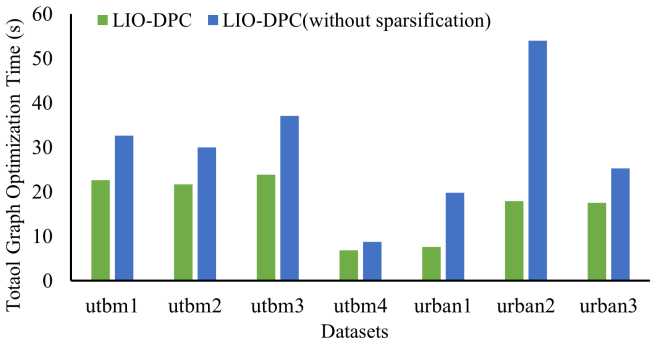

Fig. 4. Total Graph Optimization Time in Seconds for Different Dataset Sequences.

Compared to LIO-DPC (without optimization), the original LIO-DPC significantly improves system accuracy. This demonstrates that integrating filtering methods with pose graph optimization via the dynamic pose chain is both practical and efficient. Furthermore, compared to LIO-DPC (without sparsification), the original LIO-DPC not only achieves higher accuracy but also significantly reduces the total computation time of graph optimization. This indicates that our loop quality evaluation metric effectively selects high-quality loops, reducing the computational burden of graph optimization and minimizing loop closure errors.

V. CONCLUSIONS

We propose LIO-DPC, a LiDAR-inertial odometry system that integrates filtering and graph optimization via a dynamic pose chain. The pose chain is incrementally updated in real-time through filtering and batch refined with graph optimization, enabling fast and accurate pose estimation. To enhance efficiency, we introduce a loop quality metric that selects only high-quality loops for optimization, reducing computation while improving accuracy. Real-world experiments show that LIO-DPC achieves higher accuracy than state-of-the-art methods while maintaining real-time performance.

VI. ACKNOWLEDGMENT

This work was supported by the Fundamental Research Funds for the Central Universities under Grant (Project No. 2024CDJGF-032, 2023CDJXY-039, 2024CDJGF003, 2024CDJGF-019, 2023CDJKYJH026).

REFERENCES

[1] Y. Wu, T. Guadagnino, L. Wiesmann, L. Klingbeil, C. Stachniss, and H. Kuhlmann, "Lio-ekf: High frequency lidar-inertial odometry using extended kalman filters," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 13 741–13 747.

[2] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lio2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.

[3] H. Ye, Y. Chen, and M. Liu, "Tightly coupled 3d lidar inertial odometry and mapping," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 3144–3150.

[4] D. Lee, M. Jung, W. Yang, and A. Kim, "Lidar odometry survey: recent advancements and remaining challenges," *Intelligent Service Robotics*, vol. 17, no. 2, pp. 95–118, 2024.

[5] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based slam," *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2010.

[6] W. Xu and F. Zhang, "Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.

[7] C. Qin, H. Ye, C. E. Pranata, J. Han, S. Zhang, and M. Liu, "Lins: A lidar-inertial state estimator for robust and efficient navigation," in *2020 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2020, pp. 8899–8906.

[8] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and D. Rus, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2020, pp. 5135–5142.

[9] P. Geneva, K. Eckenhoff, Y. Yang, and G. Huang, "Lips: Lidar-inertial 3d plane slam," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 123–130.

[10] J. Tang, X. Zhang, Y. Zou, Y. Li, and G. Du, "A high-precision lidar-inertial odometry via kalman filter and factor graph optimization," *IEEE Sensors Journal*, vol. 23, no. 11, pp. 11 218–11 231, 2023.

[11] P. J. Besl and N. D. McKay, "Method for registration of 3-d shapes," in *Sensor fusion IV: control paradigms and data structures*, vol. 1611. Spie, 1992, pp. 586–606.

[12] T. Shan, B. Englot, C. Ratti, and D. Rus, "Lvi-sam: Tightly-coupled lidar-visual-inertial odometry via smoothing and mapping," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 5692–5698.

[13] W. Ding, S. Hou, H. Gao, G. Wan, and S. Song, "Lidar inertial odometry aided robust lidar localization system in changing city scenes," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 4322–4328.

[14] S. Zhao, Z. Fang, H. Li, and S. Scherer, "A robust laser-inertial odometry and mapping method for large-scale highway environments," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 1285–1292.

[15] Z. Wang, L. Zhang, Y. Shen, and Y. Zhou, "D-liom: Tightly-coupled direct lidar-inertial odometry and mapping," *IEEE Transactions on Multimedia*, vol. 25, pp. 3905–3920, 2023.

[16] X. Zuo, P. Geneva, W. Lee, Y. Liu, and G. Huang, "Lic-fusion: Lidar-inertial-camera odometry," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 5848–5854.

[17] N. Carlevaris-Bianco, M. Kaess, and R. M. Eustice, "Generic node removal for factor-graph slam," *IEEE Transactions on Robotics*, vol. 30, no. 6, pp. 1371–1385, 2014.

[18] A. Cunningham, M. Paluri, and F. Dellaert, "Ddf-sam: Fully distributed slam using constrained factor graphs," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010, pp. 3025–3030.

[19] V. Bichucher, J. M. Walls, P. Ozog, K. A. Skinner, and R. M. Eustice, "Bathymetric factor graph slam with sparse point cloud alignment," in *OCEANS 2015 - MTS/IEEE Washington*, 2015, pp. 1–7.

[20] J. Vallvé, J. Solà, and J. Andrade-Cetto, "Graph slam sparsification with populated topologies using factor descent optimization," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 1322–1329, 2018.

[21] P. Chauchat, A. Barrau, and S. Bonnabel, "Factor graph-based smoothing without matrix inversion for highly precise localization," *IEEE Transactions on Control Systems Technology*, vol. 29, no. 3, pp. 1219–1232, 2021.

[22] S. Bai, W. Wen, L.-T. Hsu, and P. Yang, "Factor graph optimization-based smartphone imu-only indoor slam with multi-hypothesis turning behavior loop closures," *IEEE Transactions on Aerospace and Electronic Systems*, pp. 1–21, 2024.

[23] J. Tang, Y. Chen, X. Niu, L. Wang, L. Chen, J. Liu, C. Shi, and J. Hyyppä, "Lidar scan matching aided inertial navigation system in gnss-denied environments," *Sensors*, vol. 15, no. 7, pp. 16 710–16 728, 2015.

[24] W. Zhen, S. Zeng, and S. Soberer, "Robust localization and localizability estimation with a rotating laser scanner," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 6240–6245.

[25] C. Le Gentil, T. Vidal-Calleja, and S. Huang, "In2lama: Inertial lidar localisation and mapping," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6388–6394.

[26] K. Li, M. Li, and U. D. Hanebeck, "Towards high-performance solid-state-lidar-inertial odometry and mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5167–5174, 2021.

[27] T. P. Setterfield, R. A. Hewitt, A. T. Espinoza, and P.-T. Chen, "Feature-based scanning lidar-inertial odometry using factor graph optimization," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3374–3381, 2023.

[28] Z. Yan, L. Sun, T. Krajník, and Y. Ruichek, "Eu long-term dataset with multiple sensors for autonomous driving," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 10 697–10 704.

[29] L.-T. Hsu, F. Huang, H.-F. Ng, G. Zhang, Y. Zhong, X. Bai, and W. Wen, "Hong kong urbannav: An open-source multisensory dataset for benchmarking urban navigation algorithms," *NAVIGATION: Journal of the Institute of Navigation*, vol. 70, no. 4, 2023.

[30] N. Carlevaris-Bianco, A. K. Ushani, and R. M. Eustice, "University of michigan north campus long-term vision and lidar dataset," *The International Journal of Robotics Research*, vol. 35, no. 9, pp. 1023–1035, 2016.

[31] C. Bai, T. Xiao, Y. Chen, H. Wang, F. Zhang, and X. Gao, "Faster-lio: Lightweight tightly coupled lidar-inertial odometry using parallel sparse incremental voxels," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4861–4868, 2022.